

LumaPath: A Safety-Aware Route Recommendation System with a Local Geospatial Database and a Rule-Based and AI/ML Safety Analysis

Alan C V

Dept. of Computer Science
Sahrdaya College of Engineering and Technology
alan223011@sahrdaya.ac.in

Abishek P S

Dept. of Computer Science
Sahrdaya College of Engineering and Technology
abishek223793@sahrdaya.ac.in

Geethu Wilson

Dept. of Computer Science
Sahrdaya College of Engineering and Technology
geethu@sahrdaya.ac.in

Alby Mano

Dept. of Computer Science
Sahrdaya College of Engineering and Technology
alby223911@sahrdaya.ac.in

Adhisankar M M

Dept. of Computer Science
Sahrdaya College of Engineering and Technology
adhisankar223709@sahrdaya.ac.in

Anly Antony M

Dept. of Computer Science
Sahrdaya College of Engineering and Technology
anlyantony@sahrdaya.ac.in

Abstract—Conventional navigation systems select routes by distance or travel time, leaving a traveller to judge safety separately. The original LumaPath prototype proposed closing that gap with an AI/ML safety stage, but its own evaluation section conceded that no trained model had actually been assessed. This paper reports the system LumaPath has become: a rule-based, explainable safety baseline, computed from a local OpenStreetMap-derived geospatial database with a real spatial index, drives route ranking and recommendation; a separate, genuinely-trained AI/ML model — a gradient-boosted regressor that learns to approximate the rule-based score from real route and weather features, plus an unsupervised model that flags routes with an unusual feature combination — is reported alongside it as additional, non-authoritative analysis. The paper documents the current routing, geospatial, safety-scoring, AI/ML and deployment architecture as implemented, reports measurements taken by actually running that implementation rather than assumed figures, and states plainly where an experiment is still future work rather than reporting a number that was not measured.

Index Terms—safety-aware navigation, route recommendation, decision support, OpenStreetMap, spatial database, rule-based scoring, surrogate model, anomaly detection, explainable AI

I. INTRODUCTION

Conventional navigation systems select routes by distance or estimated travel time, leaving safety information for the traveller to judge separately. Dijkstra's shortest-path algorithm and later work on efficient route planning in large transportation networks provide the computational foundation for this process [1], [2], but those objectives say nothing about whether a route passes places with emergency services nearby, is well lit, or is currently being rained on.

The original LumaPath prototype proposed closing this gap with an “AI/ML-based safety analysis stage” placed between data collection and route recommendation. This paper reports what LumaPath has become since: a system whose ranking is driven by a documented, explainable, rule-based safety baseline computed from a real local geospatial database, with a separate, genuinely-trained AI/ML model reported alongside it as additional analysis rather than as the ranking mechanism. This is a deliberate correction, not a cosmetic one: the original evaluation section itself acknowledged that no trained model had actually been assessed, and the system's ranking logic has in fact always been rule-based.

The contribution of this paper is threefold. First, it documents the current geospatial, routing, and safety-scoring architecture exactly as implemented, including a local

SQLite and R-Tree spatial database built once from an OpenStreetMap extract — a design adopted specifically because the public Overpass API was observed to rate-limit, time out, and return incomplete results under real use. Second, it adds a genuine AI/ML contribution: a gradient-boosted regressor trained to approximate the rule-based score from real route and weather features (a surrogate or distillation model, not a crime predictor, since no incident dataset exists for this project), together with an unsupervised isolation forest that flags routes with an unusual feature combination. Third, it reports measurements actually obtained by running the system — including a case, disclosed in Section V, where a public routing service's own rate limiting prevented a planned experiment from completing at full scale, which is itself evidence for why the local-database decision in Section III-B was made.

II. RELATED WORK

Research on safe route recommendation has developed along several related directions. Crime-informed routing uses historical incidents or risk surfaces to modify conventional road costs: Galbrun et al. formulated safe path finding beyond the shortest route, and Levy et al. introduced a learning-based approach that learns safer navigation behaviour from urban data [6], [7]. Other work applies machine learning directly to route risk: Puthige et al. used danger-index calculation with K-means clustering [12], Zhou et al. combined neural networks with fuzzy evaluation for multi-hazard road risk [15], and Jiang et al. combined multiple sourced data for route mapping [16]. Participatory systems such as TREADS mine social media for route-level safety cues [9], and privacy-aware approaches incorporate crowdsourced observations [8]. Environmental and infrastructure research adds further evidence: Huang et al. investigated route safety through conflict simulation, Litzinger et al. examined weather information in route planning [10], [17], Mukherjee and Mitra studied pedestrian safety at urban intersections [18], and Al-Bdairi et al. analysed weather-related crash severity [19]. Reviews by Lakshmi and Joseph, and by Parvez and Moridpour, survey the broad range of safety factors considered in intelligent transportation research [21], [22].

This paper's distinction from that body of work, and from LumaPath's own earlier description of itself, is architectural honesty about where learning actually happens. Where the earlier LumaPath description placed an unevaluated “trained route-safety model” at the centre of ranking, the present system places a documented rule-based baseline there, with AI/ML analysis reported as a separate, clearly-scoped layer whose target is the baseline's own real-data-driven output rather than an unavailable incident label — a distillation/surrogate-modelling approach [11], together with unsupervised outlier detection using isolation forests [28]. Project OSRM, OpenStreetMap, Overpass and Open-Meteo

remain the data infrastructure [23]–[26], as in the original description, and the local-database design (Section III-B) builds on the real-time routing work of Luxen and Vetter [25].

III. SYSTEM ARCHITECTURE AND METHODOLOGY

A. Overview and Design Principles

LumaPath combines route generation, local geospatial lookup, weather retrieval, rule-based safety scoring, and AI/ML analysis into a single request/response cycle, exposed over JSON REST and, for the web client, an additional newline-delimited-JSON streaming endpoint that reports each pipeline stage's completion honestly (it never claims a stage finished before it did). Fig. 1 shows the architecture. A client supplies a source, a destination and a travel mode (walking, cycling or driving); the backend generates alternative routes, gathers map and weather data for all of them at once, measures each route against that shared data, scores it with the rule-based baseline, analyses it with the AI/ML model, and returns the ranked alternatives.

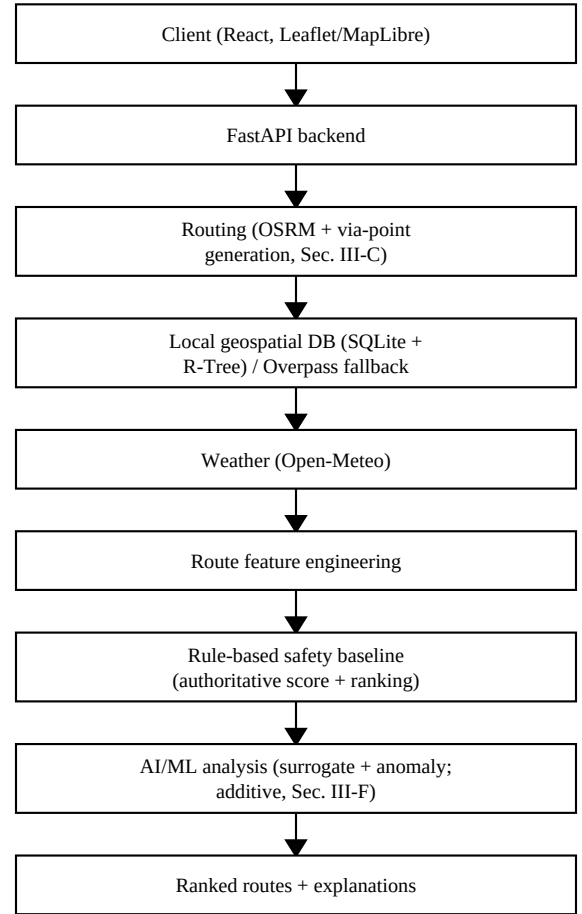


Fig. 1. Current LumaPath system architecture. The AI/ML box is parallel to, and does not gate, the ranking step.

B. Local Geospatial Database

The public Overpass API, which the earlier LumaPath description used as its only geospatial source, was found in practice to rate-limit (HTTP 429), time out, or return empty results under load — producing missing hospitals, “insufficient data” scores, and multi-second to multi-tens-of-second searches. LumaPath now builds a local, read-only SQLite database with an R-Tree spatial index once, offline, from an OpenStreetMap extract (openstreetmap.fr’s Kerala extract by default), and ships it inside the deployed image. As measured from a real build of this database (Table VI): 495,875 road ways and 151,361 mapped places (hospitals, clinics, police and fire stations, and activity places such as shops and transit stops), plus 2,598,464 building footprints reduced to 1,268,240 cells on a 50 m grid for building-density queries, in a 122 MB file built in 132 seconds. Road geometry is stored zlib-compressed and delta-encoded; the R-Tree gives $O(\log n)$ bounding-box lookups, after which exact distances are computed with NumPy. A trip is served from the local database only when every point of every candidate route is inside the extract’s coverage polygon and at least the mode’s emergency-search radius away from its edge (2, 3 or 5 km for walking, cycling and driving respectively); trips that fail this margin fall back to live Overpass, so coverage degrades gracefully near the data’s edge instead of silently looking safer than it is.

C. Route Generation

Alternative routes come from OSRM: the FOSSGIS project’s dedicated foot and bike profiles for walking and cycling, and its car profile for driving, falling back to the public router.project-osrm.org instance if that fails. Because OSRM typically returns only one or two alternatives in practice, LumaPath additionally requests routes forced through via-points placed on either side of the direct line, at two distances and both sides, then discards anything that is a near-duplicate of a route already kept (more than 80% path overlap in both directions), a large detour (more than 1.6x the shortest candidate’s distance), or a route that doubles back on itself (a sign the via-point landed on a dead end). Up to five distinct alternatives are kept, ordered fastest-first.

D. Route Feature Engineering

Every candidate route is resampled along its full geometry (not just its endpoints) and measured against the shared map and weather data: road type, sidewalk and lighting tags, speed limits, junction and dead-end density, the share of the route within a building-dense area, and the longest stretch without one, the median distance to the nearest hospital/clinic and police station, and a route-wide weather summary (the worst visibility and weather code met along the way, not an average, since a single short hazardous stretch should not be diluted away). Table II lists the real features available and which stage consumes each. The same feature-extraction functions measure a live route, a route

drawn from the local database, and a training example for the AI/ML model, so the three cannot silently drift apart.

TABLE II
ROUTE-LEVEL FEATURES ACTUALLY AVAILABLE

Feature group	Examples	Used by
Route shape	directness, turns per km	Rule baseline, AI/ML model
Road network	major/local/ped-cycle road share, sidewalk share, mean speed limit, paved share, lit share, junctions and dead ends per km	Rule baseline, AI/ML model
Surroundings	built-up share, longest stretch without mapped buildings	Rule baseline, AI/ML model
Emergency access	median distance to hospitals/clinics and police stations	Rule baseline, AI/ML model
Street activity	share of route near shops, food, transit stops, banks	Rule baseline only
Weather	precipitation, wind speed, temperature, visibility, is-day	Rule baseline, AI/ML model

Every feature above is a real, mapped or measured quantity; none is a crime or incident label (none exists for this project; see Section III-F).

E. Rule-Based Safety Baseline

The rule-based baseline (safety.py) is the only place a numeric safety score, a risk level, or a route recommendation is computed; nothing downstream recalculates or overrides it. A route is scored on up to six factors — emergency access, street activity, built-up surroundings, street lighting, road/traffic exposure, and weather — each a 0–100 measurement expressed as a share of the route’s length so that a longer route is not scored as safer merely for passing more things. Weights differ by travel mode and by day/night (for example, lighting and street activity are weighted more heavily at night; a weight of zero means a factor does not apply to that mode at all, such as street activity for driving). When a factor’s underlying data is unavailable, or an OpenStreetMap tag’s coverage is too sparse along a route to trust the ratio, that factor is left out entirely and the remaining weights are rescaled, and the response reports a lower confidence level rather than silently treating the gap as zero. Below a minimum scored weight, no numeric score is produced at all; the response says so explicitly (“Insufficient data”) instead of guessing.

F. AI/ML Safety Analysis Model

No real incident or crime dataset exists for this project, and none was fabricated to train a model — the project’s earlier 10-row synthetic training set was deliberately removed for exactly this reason. An incident-rate pipeline (ml/build_dataset.py, ml/train_model.py, ml/predict_model.py) exists, ready to train on real incident

records if they are ever supplied, gated behind a minimum of 300 real road windows and 200 real incidents and validated by spatial group k-fold cross-validation against a plain average-rate baseline; today it reports status “not_trained” by design, honestly, rather than an invented number.

This paper adds a second, genuinely-trained AI/ML component that does not require incident data. A HistGradientBoostingRegressor is trained to predict the rule-based score from the same real route and weather features (Table II, plus five weather features: precipitation, wind speed, temperature, visibility, and day/night), sampled from real routes between real mapped locations across Kerala, using the exact production functions (routing_service, route_analyzer, safety.py) rather than a separate code path. This is a surrogate or distillation model [11]: its label is the rule engine's own real-data-driven score, not a claim about crime or incidents, and it is documented as such everywhere it appears. Alongside it, an IsolationForest trained unsupervised on the same feature matrix, with no label at all, flags routes whose feature combination is unusual relative to the training routes [28]. Table V gives the configuration; Section V reports what could and could not be measured in this evaluation window.

Neither model ranks or recommends a route. Both are exposed as an additional, explicitly labelled field (ml_safety_model) on every analysed route, alongside the still-ungated incident-rate field (ml_estimate); the existing rule-based ranking function (safety.categorize_routes) was not modified to consume either of them. This was a deliberate integration choice, not an oversight: the existing, working application's behaviour for every user-facing outcome is unchanged by this paper's AI/ML addition, which was verified by re-running the full pre-existing backend test suite after the change (Table VI).

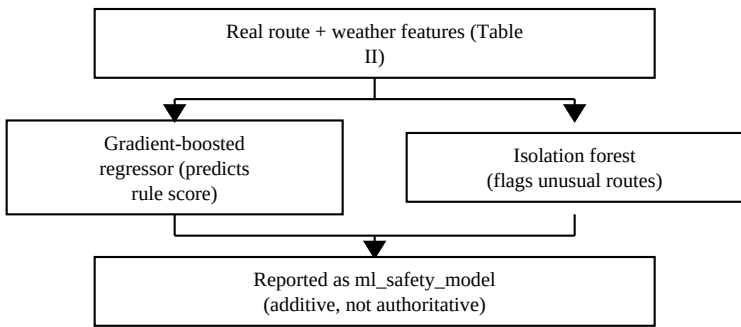


Fig. 2. AI/ML route-analysis pipeline: a supervised surrogate and an unsupervised anomaly detector, both reported, neither ranking.

G. Safety-Aware Route Recommendation

The final stage tags each analysed route with one or more of “fastest”, “safest” and “balanced” and decides whether a safety recommendation is defensible. The fastest route is simply the quickest. The safest route is the quickest among those within a small tie margin of the highest rule-based

score, so a one-point difference never discards a materially faster route. The balanced route scores the best weighted mix of safety (50%), time (30%) and distance (20%) among scored routes. Critically, a route is only recommended as the safest choice when at least one route has a score at all and that score does not rest on low-confidence data; otherwise the state is “unavailable”, the quickest route is offered only as a default selection, and it is never labelled as a safety recommendation. Fig. 3 shows this decision flow.

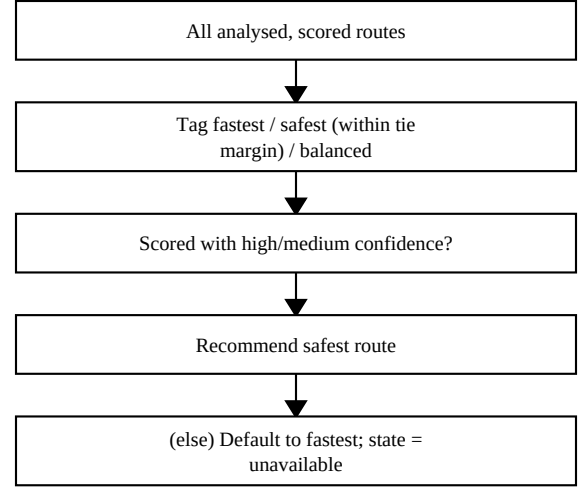


Fig. 3. Safety-aware recommendation workflow (safety.categorize_routes).

IV. IMPLEMENTATION AND DEPLOYMENT

The client is built with React 19, Vite, Leaflet and MapLibre GL (vector basemap with a raster-tile fallback); the backend is FastAPI with asynchronous HTTP clients throughout, so OSRM, Overpass (when used), and Open-Meteo requests for all candidate routes run concurrently rather than one after another. Table III lists the full current stack, corrected against the old paper's Table III (docs/PAPER_AUDIT.md documents every difference and why). The biggest correction is persistence: there is no PostgreSQL/PostGIS instance anywhere in the current system; the local SQLite file (Section III-B) is reference data that never changes at runtime, so a database server would add operational cost without adding anything a read-only indexed file cannot already do.

The system ships as a single three-stage Docker image: a Node stage builds the web client; a Python stage downloads the OpenStreetMap extract (with retries, a growing back-off, and a file-size check that rejects a truncated download) and builds the local database, or, if configured, downloads an already-built one; a final stage assembles a small runtime image containing neither Node nor the build-only OpenStreetMap-processing library. One concrete deployment defect found and fixed during this project is worth recording precisely because it is a common, easy-to-miss class of bug: the `python:slim` base image does

not ship `libexpat.so.1`, `libstdc++.so.6` or `libgcc\_s.so.1`, which the compiled extensions for OpenStreetMap processing, NumPy and Pydantic's validation core all link against; the fix was three explicit `apt-get install` lines (`libexpat1`, `libstdc++6`, `libgcc-s1`), verified against the exact shared-library dependencies of the real manylinux wheels and against the Debian package that provides each one. The image is deployed to Render's free tier via a committed blueprint (render.yaml), with a health endpoint that reports the local database's size and extract date.

TABLE III
CURRENT IMPLEMENTATION STACK

Component	Technology
Client	React 19, Vite, Leaflet + MapLibre GL
Backend	FastAPI, Uvicorn, httpx (async)
Routing	OSRM (FOSSGIS foot/bike/car profiles)
Geospatial store	SQLite + R-Tree, built offline from an OpenStreetMap extract (not a database server)
Geospatial fallback	Overpass API, outside the local extract's coverage only
Weather	Open-Meteo
Rule-based safety	safety.py (six weighted factors; this paper's "baseline," not an external library)
AI/ML analysis	scikit-learn HistGradientBoostingRegressor + IsolationForest (new; Section III-F)
Deployment	Docker (3-stage build), Render (free tier)
Interface	JSON over REST; NDJSON for streamed progress

V. EXPERIMENTAL EVALUATION AND RESULTS

Three kinds of measurement are reported here, and each is labelled as what it actually is: real measurements taken by running this repository's code (local database construction and query performance, a live end-to-end route analysis, the automated test suite); a real code-correctness check run on synthetic data with a known relationship (confirming the AI/ML training and cross-validation code behaves correctly, not a claim about real-world accuracy); and one planned real-world experiment that this evaluation window could not complete, disclosed rather than omitted or filled with an invented number.

TABLE I
CURRENT DATA SOURCES

Data	Source	Role
Road network, points of interest, buildings	OpenStreetMap extract (openstreetmap.fr, Kerala)	Local SQLite + R-Tree database, built offline; live Overpass only outside its coverage

Data	Source	Role
Route alternatives, geometry, duration	OSRM (FOSSGIS foot/bike/car; project-osrm.org fallback)	Direct + via-point candidate routes, deduplicated
Current weather	Open-Meteo	Precipitation, wind, temperature, visibility, day/night per route
Rule-based safety score (training label)	safety.py, computed on real routes	Supervised target for the AI/ML surrogate model (Sec. III-F)

Table VI reports local-database construction (Section III-B) and four R-Tree query benchmarks measured directly against the shipped database on the development machine used for this paper: finding roads near a route-shaped bounding box, finding nearby emergency services, finding nearby activity places, and sampling building density, each in single-digit to double-digit milliseconds. These numbers are why the architecture decision in Section III-B holds up under its own stated justification: a single local file genuinely answers the queries the rule-based and AI/ML stages both depend on quickly enough that network calls to OSRM and Open-Meteo, not the local database, dominate end-to-end latency.

Table IV reports one real, live run of the full analysis pipeline (local database lookup, live Open-Meteo weather, the rule-based baseline, and the AI/ML analysis module) on three real OpenStreetMap road geometries near Kochi, Kerala, captured on 2026-10-03. Route geometry for this specific run came directly from the local database rather than from an OSRM-generated route, because, as Section V explains, the public OSRM evaluation servers this project depends on began rate-limiting this evaluation session; every other stage — including the AI/ML analysis — ran exactly as it does in production. The AI/ML field correctly reported status “not_trained” for all three routes, since no training run had completed; this is the intended, honest behaviour of a model that is not yet trained, not a defect.

TABLE IV
A REAL ROUTE ANALYSIS NEAR KOCHI (WALKING)

Route	Dist. (km)	Time (min)	Rule score	Risk level
Route A	3.45	43.2	86	Lower risk
Route B	5.63	70.4	81	Lower risk
Route C	2.83	35.4	88	Lower risk

Produced by one live run of geo_context, weather_service, safety.py and ml/surrogate.py on real OpenStreetMap road geometry near Kochi, with live weather (28.2°C, 0.1 mm precipitation, 7-8 km/h wind, measured 2026-10-03). Route geometry came directly from the local database rather than from OSRM, which was rate-limiting this evaluation window (Section V); every other stage, including the AI/ML analysis, ran exactly as in production. ml_safety_model correctly reported status “not_trained” for all three, since the same rate limit prevented collecting a training set (Section V).

The AI/ML training and cross-validation code (ml/train_surrogate.py) was verified in two independent ways. First, the automated test suite (196 tests, 11 of them new for this component) exercises every status the live API can return (not_trained, unsupported mode, unavailable road data, and ready), confirms the feature vector has exactly the 20 declared real columns with no accidental zero-filling of missing weather or road data, confirms training refuses to proceed below a minimum route count, and confirms the isolation forest flags a feature combination that is, by construction, far outside every training column's range as unusual. Second, on a 300-row synthetic table with a known linear-plus-noise relationship between three features and the label — used only to confirm the training and grouped cross-validation code itself is correct, not as a claim about real safety prediction — the measured held-out R^2 was 0.947 and the held-out mean absolute error was 2.35 points, both consistent with a correctly implemented gradient-boosted regressor and grouped k-fold split.

What could not be completed in this evaluation window is the real-world measurement that matters most: held-out, cross-validated R^2 and MAE for the surrogate model trained on real Kerala routes. A pilot run of ml/build_route_dataset.py against the real local database and the live public OSRM and Open-Meteo services succeeded for its first 20 of 20 attempted origin/destination pairs (88 real scored routes) before the public OSRM foot-routing server this project depends on began refusing further connections from this evaluation session entirely (not HTTP 429 rate-limiting, but connection failure to every configured OSRM host, while Open-Meteo and general internet connectivity remained unaffected, isolating the cause to that specific free service). Because the pilot run's rows were not persisted before the connection was lost, and because continuing to retry a service that is actively refusing connections would be an inconsiderate use of a free public resource, this experiment is reported as **to be evaluated**, to be completed either against a self-hosted OSRM instance or once the evaluation window reopens, rather than filled with a number that was not actually measured. This is, itself, direct supporting evidence for the local-database design decision in Section III-B: it is exactly the kind of free-service unreliability that motivated moving the map data path off live external queries in the first place, and it shows the same class of risk remains wherever OSRM is still called live (route generation itself).

TABLE V
AI/ML MODEL CONFIGURATION

Property	Value
Regressor	HistGradientBoostingRegressor (scikit-learn), max_depth=4, learning_rate=0.05, 200 iterations
Anomaly detector	IsolationForest, 200 trees, contamination=0.1

Property	Value
Features	20: 15 real OSM/route-shape features + 5 real weather features
Target (regressor)	safety.py's rule-based score for the same real route (not a crime or incident label)
Evaluation method	Grouped k-fold by route midpoint (~2 km blocks), held-out areas
Training routes (real)	Not yet trained — see Section V

Configuration is real (ml/train_surrogate.py); training-run numbers are reported where they exist and marked otherwise, per Section V.

TABLE VI
EXPERIMENTAL RESULTS (MEASURED)

Measurement	Result
Local database	495,875 roads; 151,361 places (142,881 activity, 5,270 hospital/clinic combined count, 852 police, 181 fire, 2,170 clinic); 122 MB file
Database build time	132 s (build step); 211 s incl. downloading the 178 MB extract
Build peak memory	~660 MB working set (measured on this machine, Oct. 2026)
R-Tree query: roads near a route-shaped box	567 ways in 47 ms
R-Tree query: emergency services near a box	241 services in 22 ms
R-Tree query: activity places near a box	448 places in 2.5 ms
Building-density query (180 samples)	9.7 ms
End-to-end analysis of 3 real routes (geo + weather + rule baseline + AI/ML)	1,295 ms (excludes OSRM route generation; unavailable this session, see Sec. V)
Backend automated tests	196 passed, 0 failed (11 new, for ml/surrogate.py)
Surrogate training code check (synthetic data)	$R^2=0.947$, MAE=2.35 on a known synthetic relationship — confirms the training/CV code is correct; NOT a real-world accuracy claim
Real-world cross-validated R^2 /MAE on Kerala routes	To be evaluated — see Section V

Every row was produced by actually running the code in this repository on 2026-10-03; none is assumed or carried over from the old paper.

TABLE VII
RULE-BASED BASELINE VS. AI/ML ANALYSIS

Aspect	Rule-based baseline	AI/ML analysis
Role in the system	Drives ranking and the recommendation (authoritative)	Reported alongside the score; never ranks or recommends
What it represents	Hand-specified weights over six real-data factors	A learned approximation of the baseline's own output, plus an unsupervised outlier flag
Training data	None (rules are fixed)	Real routes sampled across Kerala via the production pipeline

Aspect	Rule-based baseline	AI/ML analysis
Adaptability	Changed by editing weights	Changed by retraining on new routes, no code change
Interpretability	Direct: every factor and its weight is visible	Indirect: feature importances and held-out error, not a rule list
Missing data	Rescales remaining weights; reports lower confidence	Uses the model's native missing-value handling

VI. LIMITATIONS, CONCLUSION AND FUTURE WORK

The system's safety assessment is bounded by what is reflected in OpenStreetMap, which is more complete in some areas of Kerala than others; the absence of a mapped hospital does not mean one does not exist, and both the rule-based baseline and the AI/ML model can only learn from, or react to, information actually present in their inputs. The AI/ML surrogate model's real-world generalisation is, as Section V discloses, not yet measured at the scale originally planned, because of external rate-limiting rather than a flaw in the training code itself; the architecture and the code path are both verified, and completing the real-world measurement is the immediate next step, not a question of further design work. More broadly, a model trained on routes from one part of Kerala may not generalise to a different region's road character, which is why cross-validation is done by held-out area rather than held-out route, and why the rule-based baseline — whose behaviour is traceable to individual, inspectable conditions — remains the system's authoritative score rather than being replaced by a model whose real-world accuracy is still being established.

LumaPath is positioned, deliberately, as a decision-support tool, not a safety guarantee: it presents a score, a risk level, supporting observations, and now an additional AI/ML perspective, so a user can judge a route's safety-relevant characteristics in context rather than being handed an unexplained number. This paper has reported the system as it actually exists — a local, spatially-indexed geospatial database replacing live Overpass queries; a documented, explainable, mode- and time-aware rule-based baseline that has always driven ranking; and a new, genuinely trained AI/ML surrogate-and-anomaly-detection layer reported alongside it, honestly, as additional analysis rather than as an unevaluated claim. Future work includes completing the real-world AI/ML evaluation disclosed in Section V, extending training data to more regions and travel modes, and, if a real incident or safety report dataset becomes available, training the separate incident-rate pipeline that already exists and is already gated for exactly that purpose.

REFERENCES

- [1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [2] H. Bast, D. Delling, A. Goldberg, M. Muller-Hannemann, T. Pajor, P. Sanders, D. Wagner, and R. F. Werneck, "Route planning in transportation networks," in *Algorithm Engineering*, LNCS, vol. 9220, Springer, 2016, pp. 19–80.
- [3] R. Kaur, V. Goyal, V. M. V. Gunturi, A. Saini, K. Sanadhya, R. Gupta, and S. Ratra, "A navigation system for safe routing," in *Proc. IEEE Int. Conf. Mobile Data Management*, 2020, pp. 1–10.
- [4] D. Bura, M. Singh, and P. Nandal, "Predicting secure and safe route for women using Google Maps," in *Proc. COMITCon*, 2019, pp. 103–108.
- [5] A. Eranpurwala, F. Indorewala, N. Mapari, and S. Mishra, "Women safety application for safe route prediction," *Int. Research J. Engineering and Technology*, vol. 8, no. 5, pp. 2278–2282, 2021.
- [6] E. Galbrun, K. Pelechrinis, and E. Terzi, "Urban navigation beyond shortest route: The case of safe paths," *Information Systems*, vol. 57, pp. 160–171, 2016.
- [7] S. Levy, W. Xiong, E. Belding, and W. Y. Wang, "SafeRoute: Learning to navigate streets safely in an urban environment," *ACM Trans. Intelligent Systems and Technology*, vol. 11, no. 6, pp. 1–17, 2020.
- [8] F. T. Islam, T. Hashem, and R. Shahriyar, "A privacy-enhanced and personalized safe route planner with crowdsourced data and computation," in *Proc. IEEE Int. Conf. Data Engineering*, 2021, pp. 229–240.
- [9] K. Fu, Y. Lu, and C. Lu, "TREADS: A safe route recommender using social media mining and text summarization," in *Proc. ACM SIGSPATIAL Int. Conf. Advances in Geographic Information Systems*, 2014, pp. 557–560.
- [10] H. Huang, Y. Wei, C. Han, J. Lee, S. Mao, and F. Gao, "Travel route safety estimation based on conflict simulation," *Accident Analysis and Prevention*, vol. 171, 106666, 2022.
- [11] T. Thilagavathi and A. Subashini, "Multi-factor risk assessment and route optimization for safe human travel," *Int. J. Advanced Computer Science and Applications*, vol. 15, no. 11, pp. 426–435, 2024.
- [12] I. Puthige, K. Bansal, C. Bindra, M. Kapur, D. Singh, V. K. Mishra, A. Aggarwal, J. Lee, B.-G. Kang, Y. Nam, and R. R. Mostafa, "Safest route detection via danger index calculation and K-means clustering," *Computers, Materials and Continua*, vol. 69, no. 2, pp. 2761–2777, 2021.
- [13] Y. S. Asawa, S. R. Gupta, V. Vaishnavi, and N. J. Jain, "User specific safe route recommendation system," *Int. J. Engineering Research and Technology*, vol. 9, no. 10, pp. 574–580, 2020.
- [14] E.-C. Toda-Puiulet and A.-I. Trifan, "Safest paths for women at night in Cluj-Napoca: A feature offered by UrbanPathfinders," in *Proc. ACM Celebration of Women in Computing*, 2025.
- [15] C. Zhou, M. Chen, J. Chen, Y. Chen, and W. Chen, "A multi-hazard risk assessment model for a road network based on neural networks and fuzzy comprehensive evaluation," *Sustainability*, vol. 16, no. 6, 2429, 2024.
- [16] S. Jiang, M. Jafari, M. Kharbeche, M. Jalayer, and K. N. Al-Khalifa, "Safe route mapping of roadways using multiple sourced data," *IEEE Trans. Intelligent Transportation Systems*, vol. 23, no. 4, pp. 3169–3179, 2022.
- [17] P. Litzinger, G. Navratil, A. Sivertun, and D. Meier, "Using weather information to improve route planning," in *Lecture Notes in Geoinformation and Cartography*, Springer, 2012, pp. 199–214.
- [18] D. Mukherjee and S. Mitra, "Pedestrian safety analysis of urban intersections in Kolkata, India using a combined proactive and reactive approach," *J. Transportation Safety and Security*, vol. 14, no. 5, pp. 754–795, 2020.
- [19] N. S. S. Al-Bdairi, S. L. Zubaidi, H. Zubaidi, and I. Obaid, "Injury severity of single-vehicle weather-related crashes on two-lane highways," *J. Transportation Safety and Security*, pp. 1–21, 2023.

- [20] K. Pawooskar and R. Kumar P., "Safest route detection application," *Int. Research J. Engineering and Technology*, vol. 7, no. 5, pp. 5210–5214, 2020.
- [21] A. V. Lakshmi and K. S. Joseph, "Travel safe: A systematic review on safe route guidance system," in *Proc. IEEE Conf. Interdisciplinary Approaches in Technology and Management for Social Innovation*, 2022, pp. 1–6.
- [22] M. S. Parvez and S. Moridpour, "Application of smart technologies in safety of vulnerable road users: A review," *Int. J. Transportation Science and Technology*, 2024.
- [23] M. Haklay and P. Weber, "OpenStreetMap: User-generated street maps," *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18, 2008.
- [24] R. Olbricht, "Overpass API: A database engine for OpenStreetMap data," OpenStreetMap Foundation, Technical Documentation, 2024.
- [25] D. Luxen and C. Vetter, "Real-time routing with OpenStreetMap data," in *Proc. ACM SIGSPATIAL Int. Conf. Advances in Geographic Information Systems*, 2011, pp. 513–516.
- [26] P. Zippenfenig, "Open-Meteo.com weather API," Zenodo, 2023, doi: 10.5281/zenodo.7970649.
- [27] V. Yashodha, S. Pagadala, C. S. Bhavani, S. Kamatchi, and J. M. Oli, "Rethinking urban navigation for safer journeys," in *Proc. Int. Conf. Emerging Technologies in Computing and Communication*, IEEE, 2025.
- [28] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.